

Apache Spark Advanced Notes

Optimization, Partitioning, & UDFs

1. Partitioning

Spark distributes data across the cluster by dividing it into partitions. A partition is a logical chunk of a large distributed data set.

- **repartition(n)** : Does a full shuffle of the data to create equal-sized partitions. Use when increasing partitions or balancing data heavily.
- **coalesce(n)** : Decreases the number of partitions without a full shuffle. It merges existing partitions. Much faster than repartition if you are only decreasing.

Example Implementation:

```
from pyspark.sql import SparkSession

# 1. Initialize SparkSession
spark = SparkSession.builder \
    .appName("PartitioningDemo") \
    .getOrCreate()

# 2. Create a sample DataFrame
# (Using range creates a quick dataset of 1 million rows)
df = spark.range(0, 1000000)

# 3. Print BEFORE partitioning
initial_partitions = df.rdd.getNumPartitions()
print("=====")
print(f"BEFORE: Initial number of partitions: {initial_partitions}")
print("=====")

# 4. Apply REPARTITION (Used to increase or evenly distribute partitions -
# causes a full shuffle)
df_repartitioned = df.repartition(10)

# Print AFTER repartition
repartition_count = df_repartitioned.rdd.getNumPartitions()
print(f"AFTER REPARTITION: Partitions changed to: {repartition_count}")
print("=====")

# 5. Apply COALESCE (Used to decrease partitions efficiently - minimizes data
# shuffling)
df_coalesced = df_repartitioned.coalesce(2)

# Print AFTER coalesce
coalesce_count = df_coalesced.rdd.getNumPartitions()
print(f"AFTER COALESCE: Partitions reduced to: {coalesce_count}")
print("=====")

# Optional: Stop the Spark session when done
spark.stop()
```

2. Optimization & Caching

Spark uses the Catalyst Optimizer to convert your code into an optimized physical plan. You can view this using `df.explain()`. To prevent recalculating data on multiple actions, you can cache/persist DataFrames in memory.

- `cache()`: Stores data in memory (MEMORY_AND_DISK for DataFrames).
- `persist()`: Lets you define exactly where to store the data (Memory, Disk, both, serialized, etc.).

Example Implementation:

```

import time
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

# 1. Initialize SparkSession
spark = SparkSession.builder \
    .appName("OptimizationAndCachingDemo") \
    .getOrCreate()

# 2. Create a slightly larger DataFrame to make the time difference noticeable
(10 million rows)
print("Generating 10 million rows...")
df = spark.range(0, 10000000).withColumn("value", col("id") % 1000)

# =====
# PART A: OPTIMIZATION (Catalyst Optimizer)
# =====
print("\n=====")
print("PART A: CATALYST OPTIMIZER PLAN")
print("=====")

# We apply multiple transformations.
# Spark's Catalyst Optimizer will combine these efficiently under the hood.
optimized_df = df.filter(col("value") > 500).filter(col("id") <
5000000).select("id", "value")

# .explain() prints the physical execution plan directly to the console/
terminal
print("Physical Execution Plan (Notice how it pushes filters down):")
optimized_df.explain()

# =====
# PART B: CACHING & PERFORMANCE TEST
# =====
print("\n=====")
print("PART B: CACHING EXECUTION TIMES")
print("=====")

# Test 1: Execution WITHOUT caching
start_time = time.time()
count_uncached = optimized_df.count() # Action triggers DAG
end_time = time.time()
print(f"1. UNCACHED Execution | Count: {count_uncached} | Time:
{round(end_time - start_time, 4)} seconds")

# Cache the DataFrame
print("\n--> Applying .cache() (This is lazy, nothing happens yet)...")

```

```

optimized_df.cache()

# Test 2: First Execution WITH caching
# (This takes time because it has to calculate the data AND write it to RAM)
start_time = time.time()
count_first_cache = optimized_df.count() # Action triggers caching
end_time = time.time()
print(f"2. FIRST CACHED Action | Count: {count_first_cache} | Time: {round(end_time - start_time, 4)} seconds (Materializing cache)")

# Test 3: Second Execution WITH caching
# (This will be lightning fast because the data is already sitting in RAM)
start_time = time.time()
count_second_cache = optimized_df.count()
end_time = time.time()
print(f"3. SECOND CACHED Action | Count: {count_second_cache} | Time: {round(end_time - start_time, 4)} seconds (Read from memory)")

# Clean up: Remove from memory when done
optimized_df.unpersist()
print("\n--> Unpersisted DataFrame to free up memory.")

spark.stop()

```

3. UDFs (User Defined Functions)

UDFs allow you to write custom Python logic and apply it to DataFrame columns.

Warning: Standard Python UDFs are slower than native Spark functions because data must be serialized between the JVM and Python.

Example Implementation:

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, udf
from pyspark.sql.types import StringType, IntegerType

# 1. Initialize SparkSession
spark = SparkSession.builder \
    .appName("BasicUDFDemo") \
    .getOrCreate()

# 2. Create a sample DataFrame
data = [("Alice", 25), ("Bob", 17), ("Charlie", 35), ("David", 15)]
df = spark.createDataFrame(data, ["Name", "Age"])

print("Original DataFrame:")
df.show()

# =====
# METHOD 1: Standard UDF using udf() function
# =====
# Define standard Python function
def categorize_age(age):
    if age >= 18:
        return "Adult"
    return "Minor"

# Convert Python function to PySpark UDF (specify return type!)
age_category_udf = udf(categorize_age, StringType())

# Apply the UDF to a column
df_method1 = df.withColumn("Category", age_category_udf(col("Age")))
print("Method 1: Standard UDF via DataFrame API")
df_method1.show()

# =====
# METHOD 2: Registering for Spark SQL
# =====
# Register the base Python function so it can be used inside SQL strings
spark.udf.register("sql_categorize_age", categorize_age, StringType())

# Create a temporary view for SQL querying
df.createOrReplaceTempView("people")

# Execute SQL query using the registered UDF
sql_df = spark.sql("""
    SELECT
        Name,
        Age,

```



```

        sql_categorize_age(Age) AS Category
    FROM people
    """)

print("Method 2: Using UDF inside Spark SQL")
sql_df.show()

# Optional: Stop the Spark session when done
spark.stop()

```

4. Docker Guide: Running the Advanced Script Headlessly

Step 1: Create Jupyter Notebook (`advanced_spark.ipynb`)

Put the code above into a notebook named `advanced_spark.ipynb`.

Step 2: Create the Dockerfile

Below is the Dockerfile configuration where the CMD executes the notebook headlessly and prints the output to the terminal:

```

FROM python:3.10-slim

# Install Java
RUN apt-get update && apt-get install -y default-jre && rm -rf /var/lib/apt/
lists/*

# Install PySpark and Jupyter
RUN pip install --no-cache-dir pyspark jupyter nbconvert

WORKDIR /app
COPY advanced_spark.ipynb .

# CMD executes the notebook headlessly and prints the output to the terminal
CMD ["jupyter", "nbconvert", "--execute", "--to", "markdown",
"advanced_spark.ipynb", "--stdout"]

```

Step 3: Build and Run

```
# Build Image
docker build -t spark-advanced-headless .

# Run Container (Watch the results print in your terminal!)
docker run --rm spark-advanced-headless
```